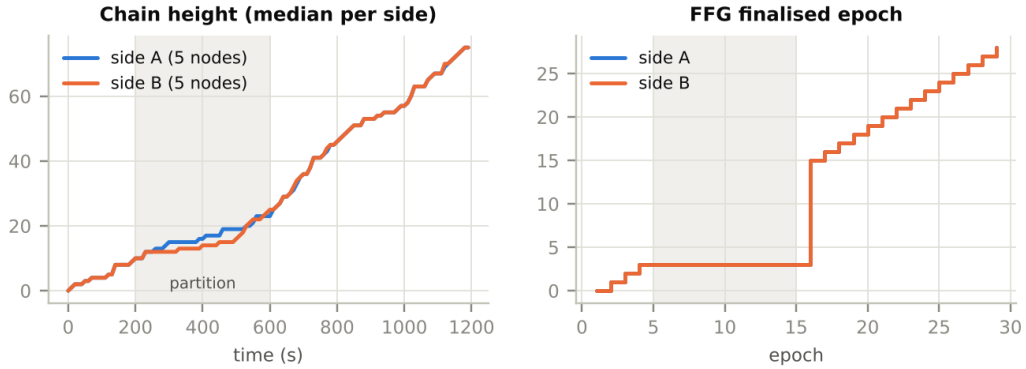
# Partition Playground — scripted adversaries on the Lab 03 network

A scenario-driven laboratory around Lab 03's *consensus* package. The engine subclasses Lab 03's `Network`/`Node` (gossip, heaviest-chain fork choice, `getchain` reconciliation, exponential-race mining are inherited unchanged) and runs one Lab 03 `pos.BeaconState` per node as the Casper FFG gadget, voting over the same gossip network. Lab 03's asyncio code runs on a **virtual-clock event loop**, so every scenario is deterministic from its seed (byte-identical event streams across processes and `PYTHONHASHSEED`s) and the full test suite runs in ≈ 21 s. Scenarios are YAML (network, adversaries, timeline, *expect*); the engine emits a JSON event stream that feeds metrics, tests and the dashboard. Settings unless stated: 10 honest nodes, 3 peers each, mean block time T = 10 s, one-hop latency 0.5 s (jitter U(0.5, 1.5)), 10 validators × 32 ETH, 40 s epochs.

| Scenario | Expected (asserted in tests) | Observed | |
|---|---|---|---|
| 50/50 partition + heal | both sides keep mining; reorg ≥ 5 on heal; converge ≤ 120 s; no finality during split; no conflicting finality | reorg 13, converged after 2 s; finality frozen at epoch 3; 0 conflicts | ✓ |
| 70/30 partition | majority reorg ≤ 2, minority ≥ 5; majority finalises every epoch; minority frozen | majority 1, minority 9; finalised epoch at heal: 70 % side 16, 30 % side 3 | ✓ |
| Eclipse of node9 | payment ≥ 6 confs then 0 (double spend); victim reorg ≥ 6; victim finality lag +1/epoch | peak 15 confs → 0; reorg 17; lag up to 15 epochs | ✓ |
| High latency (sweep) | stale rate strictly increasing in Δ/T, within 0.6–1.1 × analytic | 1.5% → 4.0% → 8.1% → 14.7% → 25.5% | ✓ |
| Orphan rate, Δ = 0.1 T | stale rate in documented band [7.0 %, 12.0 %] around 9.52 % | 8.02 % ± 0.45 (n = 3703) | ✓ |
| FFG, 40 % stake offline | leak starts epoch 14; justified again 33; finalised again 34 (bare Lab 03 BeaconState prediction) | leak 14, justified 33, finalised 34; lag peaked at 25 | ✓ |
| Extras | delayed relay raises stale rate ≥ 3 pp; equivocation evidence at all nodes ≤ 10 s; long partition → conflicting finality | 7.2 % → 16.0 %; evidence at 9 nodes within 1.9 s; 3 conflicting epochs | ✓ |

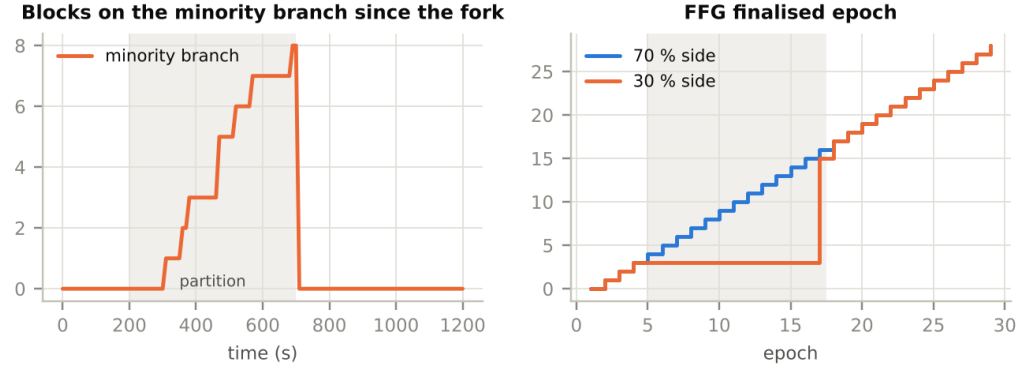
## 1 · 50/50 partition and heal




**Expected vs observed.** During the 400 s split both halves kept producing blocks (side A 10→23, side B 10→24): 1040 messages were silently dropped. At the heal Lab 03 nodes announced their heads and pulled the peer's chain (`getchain`); the lighter half adopted the heavier branch and un-confirmed 13 blocks, and all tips agreed 1.8 s after the heal (with equal heights the tie would persist until the next block). FFG: neither half holds > 2/3 of the stake, so no checkpoint was justified — finality stayed at epoch 3 on both sides, then resumed (epoch 28 at the end).

**CAP / safety–liveness.** A partition forces the choice. Nakamoto PoW picks *availability*: each side accepts transactions and confirms them, and consistency is restored only by discarding one side's history — its confirmations were never safe. FFG picks *consistency*: with no 2/3 quorum reachable it stops finalising (loses liveness) rather than finalise two histories. FLP is visible too: a partitioned node cannot tell a dead peer from a slow one, so no deterministic protocol can both always decide and never decide wrongly.

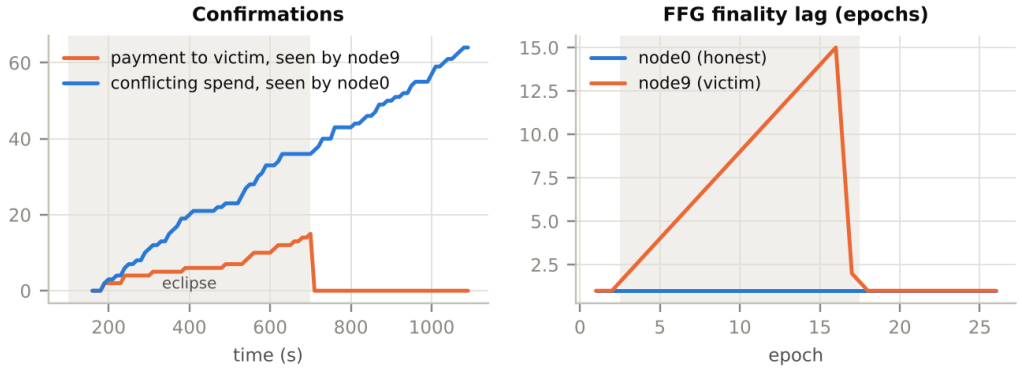
## 2 · 70/30 partition



**Expected vs observed.** The 70 % side kept the supermajority of stake, so it finalised every epoch throughout (epoch 16 at heal) and its chain only saw ordinary forks (max reorg 1). The 30 % side kept mining at 30 % of the rate — its private branch grew to 8 blocks (left) — but its finality froze at epoch 3; at the heal it lost its whole 9-block branch and adopted the majority's justification proofs (verified against its own balances), catching up on finality within one epoch.

**Interpretation.** Safety and liveness both survive on the side that holds > 2/3 of the stake; the minority keeps *availability without finality*. An operator on the minority side sees exactly the signal that matters: the PoW chain advances, the finalised checkpoint does not.

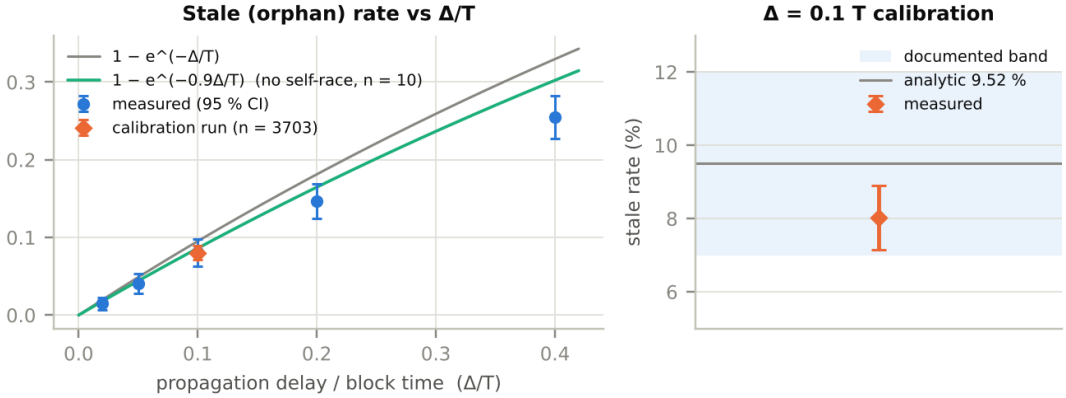
## 3 · Eclipse of one node




**Expected vs observed.** Two attacker nodes (17 % of hash power together) took all of node9's connection slots for 600 s and fed it a private chain. A payment to node9 reached 15 confirmations on that chain while the honest chain confirmed a conflicting spend of the same coins; when the eclipse ended node9 reorganised 17 blocks and the payment fell to 0 confirmations — a successful double spend against a node that followed the '6 confirmations' rule. Its FFG view saw no votes, so its finality lag grew by one per epoch (to 15); afterwards it adopted the honest proofs and jumped to epoch 17.

**Interpretation.** An eclipse is a *targeted partition*: the synchrony assumption behind PoW security (honest blocks reach you within Δ) is violated for one node only, so heaviest-chain is evaluated over an adversarial subset and k confirmations certify nothing. FFG stays *safe* for the victim — the attackers have no stake, so it never finalises their chain — but loses liveness, which makes finality lag a cheap, high-signal eclipse alarm (the dashboard raises it when one node is alone on a branch and ≥ 4 epochs behind).

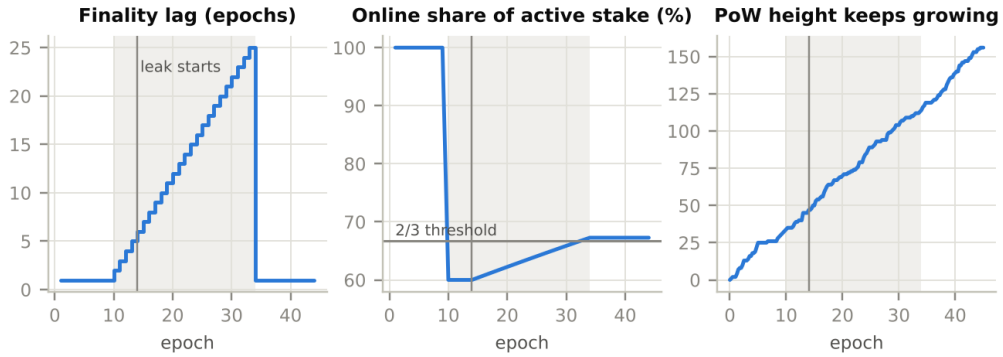
## 4 · Latency and the orphan rate




**Expected vs observed.** On a full mesh (one hop) with ten equal miners the stale rate rises monotonically with Δ/T (1.5 %, 4.0 %, 8.1 %, 14.7 %, 25.5 % for Δ/T = 0.02…0.4). Calibration at Δ = 0.1 T: measured 8.02 % ± 0.45 pp over 3703 blocks. The documented band is the analytic 9.52 % ± 2.5 pp, budgeted as a −1.0 pp systematic term (a miner never races its own block: 1 − E[e^(−0.9Δ/T)] = 8.57 %) plus ±3σ sampling noise; the measurement is within 3σ of the refined estimate as well. At large Δ/T the simple formula overestimates, because chains of forks resolve together. A relay that holds others' blocks for 4 s on the only bridge between two clusters lifted the stale rate from 7.2 % to 16.0 %.

**Interpretation.** Under partial synchrony, Δ/T is the security parameter of Nakamoto consensus: stale blocks are wasted honest work, so the fraction of hash power needed to out-race the network falls below 50 % as Δ/T grows, and a well-connected miner (or a delaying relay) gains an edge — the selfish-relay effect.

## 5 · Casper FFG with 40 % of the stake offline




**Expected vs observed.** Four of ten validators stopped attesting after epoch 9. 60 % < 2/3, so nothing was justified and the finality lag grew by exactly one per epoch (to 25) while PoW blocks kept coming (height 35 at epoch 10 → 113 at epoch 33). Lab 03's inactivity leak engaged once more than 4 epochs had passed since the last finalisation (epoch 14) and drained each absent validator by 1/64 per epoch; when the online share crossed 2/3 the link 9 → 33 justified, and epoch 34 finalised its predecessor. These epochs are asserted and equal those of a bare Lab 03 BeaconState with no network, so gossip added no error.

**Interpretation.** FFG is safety-first: it refuses to finalise without a 2/3 quorum. The leak is the liveness escape hatch — it redefines the quorum by shrinking absent stake. In this outage that is benign. Run under a *partition* instead, each side leaks the other: in a 55-epoch 50/50 split both halves eventually finalised different checkpoints (3 conflicting epochs; nodes node0, node1, node2, node3, node4 later saw their finalised checkpoints reverted by the PoW reorg on heal). Liveness under long partitions is bought with safety — CAP applies to finality gadgets too.

**Stretch: equivocation.** node3 sent conflicting attestations for epoch 5 to different halves of its peers. The double-vote evidence travelled through gossip and reached all 9 other honest nodes between 1.06 s and 1.92 s after the act; every view applied Lab 03's slashing (1/32 + correlated penalty) and finality continued on the remaining 90 %.

**Limitations.** Hash power and stake are co-located per node; checkpoint choice uses a fixed age margin rather than LMD-GHOST; Lab 03's PoW fork choice is finality-unaware, so reverted finality is measured, not prevented; latency is i.i.d. per message.